📈 SISTEMA MULTI-AGENTE IA

Informe Diario de la Bolsa Española

IBEX 35 · Mercado Continuo · BME Growth

> Plan de Proyecto & Arquitectura Técnica — Claude Pro + Claude Code · 2026
> *Documento de planificación técnica para migración de Cursor a Claude Code*

---

1. Resumen Ejecutivo

El objetivo es construir un sistema automatizado de cuatro agentes de IA que genere cada día laborable, al cierre del mercado español (17:35h), un informe completo en PDF sobre la evolución de IBEX 35, Mercado Continuo y BME Growth, y lo entregue por email sin necesidad de tener el ordenador encendido.

¿Por qué multi-agente?

- Cada agente tiene un contexto reducido y especializado → menor consumo de tokens
- Cada agente lleva sus propias SKILL.md → instrucciones precisas sin ruido
- El Agente Líder actúa como QA → valida cada paso antes de continuar
- Sistema modular: se puede mejorar un agente sin tocar los demás
- Trazabilidad clara: si algo falla, se sabe exactamente en qué etapa

---

2. Stack Tecnológico

2.1 Herramientas de desarrollo

Herramienta	Uso	Coste
Claude Pro	Acceso a Claude Code y modelos avanzados	~20 €/mes
Claude Code	Agente de codificación en terminal / VS Code	Incluido en Pro
Python 3.11+	Lenguaje principal del sistema	Gratuito
VS Code	IDE con terminal para Claude Code	Gratuito
Git + GitHub	Control de versiones y GitHub Actions	Gratuito

2.2 Librerías Python

Librería	Función	Instalación
yfinance	Descarga datos OHLC de IBEX, Continuo, BME Growth	pip install yfinance
feedparser	Parseo de RSS de Expansión y Cinco Días	pip install feedparser
pandas	Manipulación y análisis de datos	pip install pandas
matplotlib / plotly	Generación de gráficas del informe	pip install matplotlib plotly
reportlab	Creación del PDF final	pip install reportlab
smtplib	Envío del email (incluido en Python stdlib)	Sin instalación
python-dotenv	Gestión segura de credenciales	pip install python-dotenv
schedule	Programador de tareas local (alternativa)	pip install schedule

2.3 Fuentes de datos

Mercado	Fuente	Ticker / URL	Coste
IBEX 35	yfinance	^IBEX	Gratis
Mercado Continuo	yfinance	SAN.MC, TEF.MC…	Gratis
BME Growth	yfinance	Tickers BME	Gratis
Noticias	RSS Expansión	expansion.com/rss	Gratis
Noticias	RSS Cinco Días	cincodias.elpais.com/rss	Gratis

3. Arquitectura del Sistema

El sistema sigue un patrón de orquestación secuencial con validación. El Agente Líder activa cada agente en orden, valida su output antes de pasar al siguiente, y solo firma el informe final si todo el proceso ha sido correcto.

```
┌──────────────────────────────────────┐
│   👑 AGENTE LÍDER (Orchestrator + QA) │
└──────────────────────────────────────┘
                  │
                  ▼
        ┌──────────────────┐
        │ 1️⃣ Agente Researcher │
        │ Descarga OHLC + RSS  │
        │ → /data/raw/         │
        │ ✅ Líder valida      │
        └──────────────────┘
                  │
                  ▼
        ┌──────────────────┐
        │ 2️⃣ Agente Analyst    │
        │ Análisis + métricas  │
        │ → /data/analysis/    │
        │ ✅ Líder valida      │
        └──────────────────┘
                  │
                  ▼
        ┌──────────────────┐
        │ 3️⃣ Agente Writer     │
        │ PDF + gráficas       │
        │ → /output/           │
        │ ✅ Líder valida      │
        └──────────────────┘
                  │
                  ▼
        ┌──────────────────┐
        │ 📧 Envío por email   │
        │ + log del proceso    │
        └──────────────────┘
```

---

4. Ficha de Cada Agente

👑 Agente Líder — Orchestrator + QA Master

Responsabilidades:

- Lanza los agentes en secuencia correcta
- Valida output de cada agente antes de continuar
- Gestiona errores y reintentos (máx. 3)
- Envía email con PDF adjunto
- Genera log diario del proceso completo
- Alerta por email si algo falla

Skills (.md):

- `orchestrator_rules.md`
- `qa_checks.md`
- `email_sender.md`
- `error_handling.md`

🔍 Agente Researcher — Recopilador de Datos

Responsabilidades:

- Descarga OHLC diario con yfinance
- Cubre IBEX 35, Mercado Continuo, BME Growth
- Parsea RSS de Expansión y Cinco Días
- Filtra noticias relevantes por palabras clave
- Guarda CSVs y JSON en `/data/raw/YYYY-MM-DD/`
- Genera resumen de datos recopilados

Skills (.md):

- `data_sources.md`
- `yfinance_usage.md`
- `rss_parser.md`
- `file_naming.md`

📊 Agente Analyst — Analista Financiero

Responsabilidades:

- Lee datos crudos del Researcher
- Calcula variaciones, máximos, mínimos del día
- Identifica los 5 mejores y 5 peores valores
- Calcula volumen, RSI básico, tendencia
- Resume noticias con impacto en mercado
- Genera conclusiones y perspectiva del día

Skills (.md):

- `analysis_framework.md`

- `metrics_definition.md`
- `news_relevance.md`
- `output_schema.md`

✍️ Agente Writer — Redactor e Infografista

Responsabilidades:

- Lee análisis del Analyst
- Redacta resumen ejecutivo del día
- Genera gráfica de evolución IBEX 35
- Genera tabla de ganadores/perdedores
- Genera gráfica de sectores
- Maqueta y exporta PDF con reportlab

Skills (.md):

- `report_template.md`
- `chart_styles.md`
- `pdf_layout.md`
- `writing_style.md`

---

5. Estructura de SKILL.md por Agente

Las SKILL.md son el 'manual de instrucciones' de cada agente. Reducen el contexto necesario y mejoran la precisión de cada tarea. Se ubican en la carpeta del proyecto y Claude Code las lee al inicio de cada sesión.

Estructura de carpetas del proyecto

```
bolsa-agente/
├── skills/
│   ├── orchestrator_rules.md      ← Agente Líder
│   ├── qa_checks.md               ← Agente Líder
│   ├── email_sender.md            ← Agente Líder
│   ├── error_handling.md          ← Agente Líder
│   ├── data_sources.md            ← Researcher
│   ├── yfinance_usage.md          ← Researcher
│   ├── rss_parser.md              ← Researcher
│   ├── file_naming.md             ← Researcher
```

```
│   ├── analysis_framework.md     ← Analyst
│   ├── metrics_definition.md     ← Analyst
│   ├── news_relevance.md         ← Analyst
│   ├── output_schema.md          ← Analyst
│   ├── report_template.md        ← Writer
│   ├── chart_styles.md           ← Writer
│   ├── pdf_layout.md             ← Writer
│   └── writing_style.md          ← Writer
├── agents/
│   ├── leader.py
│   ├── researcher.py
│   ├── analyst.py
│   └── writer.py
├── data/
│   ├── raw/YYYY-MM-DD/
│   └── analysis/YYYY-MM-DD/
├── output/
│   └── informe_YYYY-MM-DD.pdf
├── logs/
├── .env                          ← credenciales (no subir a Git)
├── .gitignore
├── requirements.txt
├── CLAUDE.md
└── main.py                       ← punto de entrada
```

5.1 Plantilla estándar de cada SKILL.md

```markdown
```

```
# Nombre del Skill

## Objetivo
Una frase concisa de qué hace este agente/función

## Inputs esperados
Qué archivos o datos recibe

## Outputs esperados
Qué archivos o datos debe producir, con formato exacto

## Reglas obligatorias
Lista de restricciones que NUNCA debe incumplir

## Ejemplos
Ejemplo de input → output correcto

## Errores comunes a evitar
Lista de fallos frecuentes y cómo prevenirlos
```

---

6. Opciones de Entrega sin Ordenador Encendido

⭐ Opción A — GitHub Actions (Recomendada)

Ventajas:

- ✅ Totalmente gratuito (2.000 min/mes en cuenta gratuita)
- ✅ Sin servidor que gestionar
- ✅ El código ya está en GitHub → un archivo .yml activa el cron
- ✅ Logs accesibles desde el navegador

Limitaciones:

- ⚠️ Límite: 6 horas por ejecución (más que suficiente)
- ⚠️ Secretos como email/password se guardan en GitHub Secrets (seguro)

Ejemplo `.github/workflows/informe.yml`:

```yaml
```

```yaml
on:
  schedule:
    - cron: '35 15 * * 1-5'   # 17:35h Madrid (UTC+2 en verano = 15:35 UTC)

jobs:
  generate-report:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-python@v4
      - run: pip install -r requirements.txt
      - run: python main.py
        env:
          EMAIL_USER: ${{ secrets.EMAIL_USER }}
          EMAIL_PASS: ${{ secrets.EMAIL_PASS }}
```

Opción B — PythonAnywhere

- ✅ Plan gratuito suficiente para este proyecto
- ✅ Programador de tareas integrado con interfaz web
- ✅ Soporte nativo de Python, sin configuración de servidor
- ⚠️ En plan gratuito, acceso externo limitado (yfinance funciona bien)

Opción C — Raspberry Pi (local, siempre encendida)

- ✅ Control total del entorno, sin limitaciones de tiempo ni red
- ✅ Coste único ~40€ + electricidad mínima
- ⚠️ Requiere configuración inicial de cron en Linux
- ⚠️ Si se va la luz o hay problemas de red, el proceso falla

---

Comparativa de opciones

Opción	Coste	Complejidad	Fiabilidad	Recomendado para
GitHub Actions	Gratis	⭐ Baja	⭐⭐⭐⭐⭐ Muy alta	**Empezar aquí**

Opción	Coste	Complejidad	Fiabilidad	Recomendado para
PythonAnywhere	Gratis / ~5€/mes	⭐⭐ Media	⭐⭐⭐⭐ Alta	Dashboard web
Raspberry Pi	~40€ único	⭐⭐⭐ Media	⭐⭐⭐ Media	Control total offline
VPS (DigitalOcean)	~5-10€/mes	⭐⭐⭐ Media	⭐⭐⭐⭐⭐ Muy alta	Escalar el proyecto

---

7. Roadmap de Desarrollo

Fase	Semana	Objetivo	Entregable
0 · Setup	1	Instalar Claude Code, crear proyecto, estructura carpetas	Entorno listo
1 · Researcher	1-2	Descargar OHLC + RSS, guardar en /data/raw/	researcher.py funcionando
2 · Analyst	2-3	Leer datos, calcular métricas, generar análisis JSON	analyst.py funcionando
3 · Writer	3-4	Generar PDF con gráficas a partir del análisis	PDF de prueba correcto
4 · Líder + QA	4-5	Orquestar flujo completo, validaciones, logs	Sistema completo local
5 · Email	5	Envío automático del PDF por email (Gmail SMTP)	Email recibido
6 · SKILL.md	5-6	Redactar todas las SKILL.md de cada agente	Documentación completa
7 · GitHub Actions	6	Subir a GitHub, configurar cron automático	Sistema en la nube
8 · Ajuste fino	7-8	Mejorar análisis, diseño PDF, manejo de errores	v1.0 estable

---

8. Próximos Pasos Inmediatos

1. ✅ Activar Claude Pro en claude.ai

2. ✅ Instalar Claude Code
3. ✅ Conectar con VS Code
4. Ejecutar `/init` en Claude Code para crear el CLAUDE.md
5. Crear la estructura de carpetas del proyecto
6. Construir el Agente Researcher (primera sesión de código)

> 💡 **Consejos para trabajar con Claude Code**
>
> - En cada sesión, empieza con: *"Lee el archivo skills/SKILL_NAME.md antes de empezar"*
> - Trabaja en un único agente por sesión para mantener el contexto limpio
> - Usa `.env` para todas las credenciales y añádelo al `.gitignore` antes del primer commit
> - Testea cada agente de forma independiente con datos de un día concreto antes de integrar
> - Usa `/clear` entre tareas distintas para no contaminar el contexto

---

Documento generado con Claude Sonnet 4.6